DS/CMPSC 442: Artificial Intelligence
Fall 2026
Project-2 Adversarial Search
(Due: Oct 16th 11:59 PM)
Total Marks: 20

Project Description

In this project, you will need to implement search algorithms learned during the class to solve two problems. All implementation should be finished through **Python 3.9**

Files to Submit: Submit a .zip file containing exactly two files: (See Table.1 for file name and explanation).

Deliverable	Explanation
File-1: tictactoe-ai.zip	A .zip file containing your modified python files. (You will receive a zip file containing a python implementation of a tic-tac-toe agent, which you will modify. Described below.)
File-2: solution_q2.py	Implementations for all Q2 coding tasks. Must run with python solution_q2.py.

Table 1. Submission files for Project-2

Evaluation: Your code will be graded manually by TA for technical correctness. To foster the grading process, they will directly check the output printed in the terminal after running the command "python solution_q2.py". So make sure your code could be running in the terminal smoothly before making the submission.

Academic Dishonesty: We will be checking your code against other submissions in the class for logical redundancy. If you copy someone else's code and submit it with minor changes, we will know. At the same time, do not attempt to use tools like ChatGPT, LLaMa, Gemini, or any other large language model. Our cheating detectors are quite hard to fool, so please don't try. We trust you all to submit your own work only; please don't let us down. If you do, we will pursue the strongest consequences available to us.

Question 1: Designing Wild Tic-Tac-Toe Agents

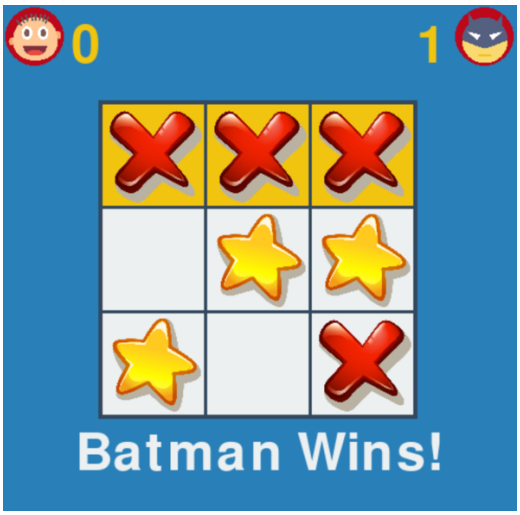


Figure.1 An example of playing tic-tac-toe

Question Description

We all know how to play tic-tac-toe. In this project, we will be implementing a variation of classic tic-tac-toe (let's call that wild tic-tac-toe). In wild tic-tac-toe, the objective is to make three in a row without allowing your opponent to make three in a row on the next move. If a player makes three in a row, the opponent gets one more move to make three in a row themselves. If the opponent can do so, they win immediately, and the first player loses. Otherwise, the first player to make three in a row wins.

The game is played on the same 3x3 grid as the traditional version. Players take turns placing their respective symbols, commonly "X" and "O", on the board. **However, the twist is that while in standard tic-tac-toe players aim to create a line of three of their symbols to win, in our modified tic-tac-toe, doing so results in a loss. This necessitates an entirely different strategy, as players must avoid making lines for themselves while trying to force their opponent into doing so.** This game often leads to more complex strategies and counter-strategies, as players must constantly evaluate the risk and reward of each move. It introduces an intriguing twist to the familiar game and challenges players to think in a counterintuitive way. Luckily for me, I have my DS/CMPSC 442 students who can help me in designing an agent that plays this wild version of tic-tac-toe for me.

Question 1.0 Setup Local Files (0 marks)

Instead of building a wild tic-tac-toe player from scratch, we will take an existing implementation that implements a minimax search based (classical) tic-tac-toe solver, and then we will modify it to solve the wild tic-tac-toe game.

As a first step, use these instructions to setup your local machine. You will need **Python 3.9**. Older versions of python will throw errors; these files are not backwards compatible.

1. Download the tictactoe-ai-master.zip file and Extract All.

2. Run *pip install pygame*
3. In order to run the game, navigate to the directory with main.py and write the command on terminal or cmd: *python main.py*

Note: You might run into some issues of missing packages, if you do run into them, you can install those missing packages first by running *pip install <package name>.*

To check whether you have successfully installed the dependencies, you can try playing a game of tic-tac-toe using the GUI application that pops up when run *python main.py*.

Question 1.1 (10 marks)

Update the engine.py file (in tictactoe-ai/tictactoe/engine.py) and board.py (in tictactoe-ai/tictactoe/board.py) file such that you can end up getting a tic-tac-toe AI solver that plays the wild version of tic-tac-toe.

Send us a .zip file containing your modified AI that plays the wild version of tic-tac-toe. We will play your AI agent to verify that your agent is playing wild tic-tac-toe in the correct way. We will play against your AI agent to see how well it plays.

Question- 2: Designing Agents for Territory Control Strategy Game

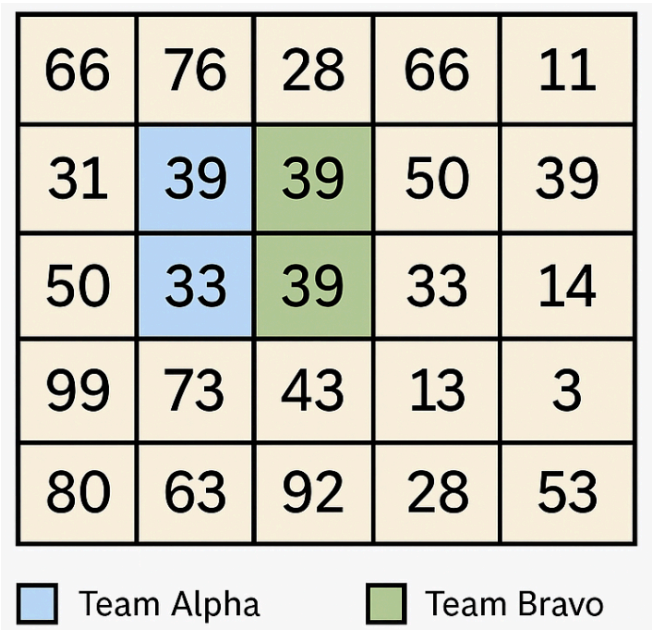


Figure.2 An example of territory control strategy game

Question Description

This project centers on developing an intelligent agent capable of playing a turn-based strategy game involving territorial acquisition on a 6×6 grid. The game simulates a simplified battlefield where two competing factions—**Team Alpha** and **Team Bravo**—take turns deploying and maneuvering units to seize control of high-value zones. The core objective is to **maximize the cumulative value of occupied territory** by the game's end.

Game Setup

- The playing field is a 6x6 grid (36 tiles), with each tile assigned a fixed strategic value between 1 and 99.
- The tile values represent varying importance—such as resource richness or tactical advantage—and remain unchanged throughout the game.
- Two teams, **Alpha** (starting first) and **Bravo**, alternate turns.
- The game concludes once all tiles have been claimed by one of the two teams.
- Movement is limited to vertical and horizontal directions; diagonal captures are not allowed.
- The winner is determined by the **sum of the point values of tiles controlled** at the end of the game.
-

Movement Options

Each turn, a player can perform **one of two types of operations**, either deploying a new unit or advancing from an existing one:

1. Strategic Deployment

- A unit can be placed on **any unoccupied tile** on the grid.
- This action does **not** affect neighboring enemy tiles—it simply expands the team's presence.
- It may be used as many times as desired across the game but only once per turn.
- Deployments are ideal for claiming isolated high-value zones.

Example: In Figure 3, Team Bravo executes a Strategic Deployment on tile [C,3] worth 39 points. The team's score increases accordingly, without impacting any of Team Alpha's existing positions.

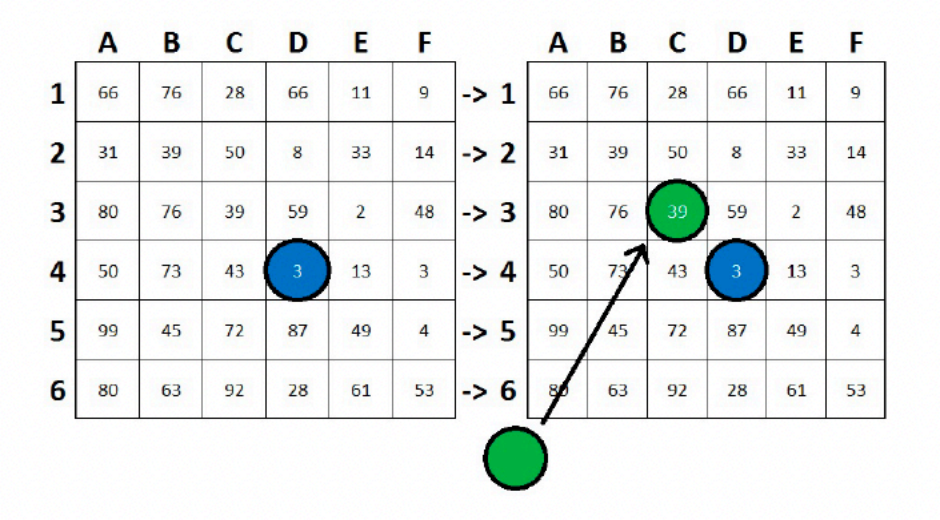


Figure.3 An example of strategic deployment

2. Tactical Assault

- From an already controlled tile, a unit may move **one tile orthogonally** (up, down, left, or right) into an **unoccupied space**.
- This move creates a **new unit** on the target tile while keeping the original position intact.
- If **any enemy tiles are directly adjacent** to the new position, they are **automatically converted** to the attacking team's control.
- This type of move **must result in a capture if possible**—aggressive conversion is mandatory when viable.

Example-1: In Figure.4, Team Bravo advances from [D,4] to [D,3], triggering a Tactical Assault that captures the adjacent enemy tile at [D,2]. If another move (e.g., to [E,4]) had been made instead with no adjacent enemies, no conversion would have occurred. Note, the score before this movement was Bravo 46 : Alpha 157 but afterwards is Bravo 113 : Alpha 149;

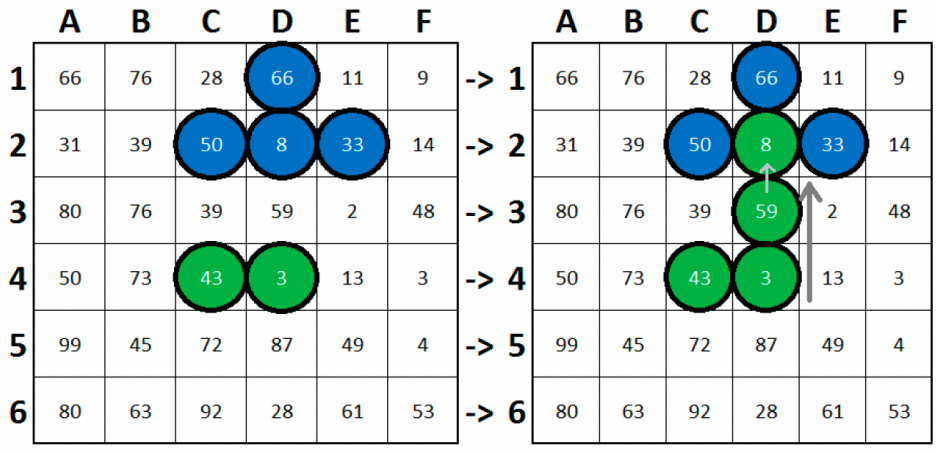


Figure.4 Example-1 of Tactical Assault

Example-2: In Figure-5, Team Alpha carries out a Tactical Assault from C2 to C3, establishing a new unit. This new position is adjacent to two Team Bravo units at D3 and C4. Both are immediately **converted to Team Alpha's control**, as they border the newly placed piece. This move demonstrates a **multi-tile conversion**, leveraging positional advantage for maximum impact. However, in the subsequent turn, **Team** Bravo has limited **response options**. Its unit at **D4** may still attempt a Tactical Assault if any adjacent tiles are available, but **the unit at D2 is now blocked in, with no neighboring unoccupied tiles, making further expansion impossible.**

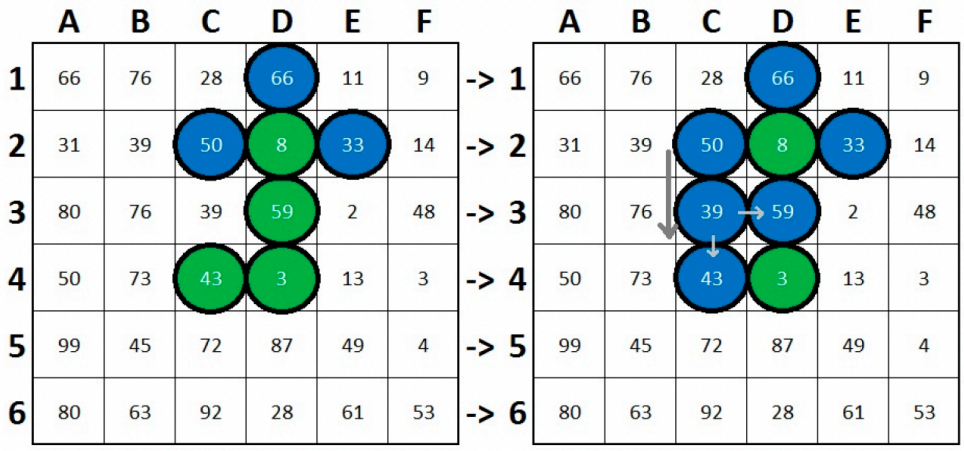


Figure.5 Example-2 of Tactical Assault

Question 2.1 (10 marks)

Implements a minimax search based AI agent for playing this game inside the solution_q2.py. It takes the Game boards as input and print out the info in the terminal (without generate any output files)

Game boards are specified through input.txt file containing 36 integers, each representing the value of a grid tile in row-major order. An example of input.txt is below:

```
66,76,28,66,11,9
31,39,50,8,33,14
80,76,39,59,2,48
50,73,43,3,13,3
99,45,72,87,49,4
80,63,92,28,61,53
```

Information printed in the terminal should contains (sample output is below):

- in each step, print the action that the team takes;
- in each step, print the resulting game board.

Expected terminal output format:

Move 1:
Minimax action for team: <Team-Alpha>
<66,76,28,66,11,9 ****////Sample Game Board
31,39,50,8,33,14
80,76,39,59,2,48
50,73,43,3,13,3
99,45,72,87,49,4
80,63,92,28,61,53>
Total score - Alpha: <number>
Total score - Bravo: <number>

Move 2:
Minimax action for team: <Team- Bravo >
<66,76,28,66,11,9 ****////Sample Game Board
31,39,50,8,33,14
80,76,39,59,2,48
50,73,43,3,13,3
99,45,72,87,49,4
80,63,92,28,61,53>
Total score - Alpha: <number>
Total score - Bravo: <number>
……
Move <N>:
Minimax action for team: <Team-Bravo >
<66,76,28,66,11,9 ****////Sample Game Board
31,39,50,8,33,14
80,76,39,59,2,48
50,73,43,3,13,3
99,45,72,87,49,4
80,63,92,28,61,53>
Total score - Alpha: <number>
Total score - Bravo: <number>

The Winner is: <Team name >